
Renforcement Java – TP

Exercice 1

En s'inspirant du programme `Sphere`, rédiger un programme `Parallelepiped` qui calcule la surface et le volume de parallélépipèdes rectangles de diverses dimensions. Le programme doit afficher la surface et le volume de parallélépipèdes de dimensions $12 \times 3 \times 5$ et $4 \times 34 \times 10$.

Exercice 2

1e partie

Rédiger un programme `TestAverage` qui calcule la moyenne des 3 nombres entiers (de type **int**) pour les triplets de nombres suivants :

3, 1, 1
-12, 2, 12
4, 6, 7

2e partie

Calculer la moyenne de nombres passés en argument au programme sur la ligne de commande. *Indication : on pourra utiliser `Integer.parseInt(arg[i])`.*

Exercice 3

On désire calculer la surface de couronnes délimitées par les cercles concentriques de rayons `r1` et `r2`. Les rayons peuvent être quelconques : `r1` peut être supérieur, égal, inférieur à `r2`. La surface ne doit pas être négative.

Pour cela on décide de définir les fonctions suivantes :

- **surfaceOfCircle** : surface d'un cercle de rayon donné
- **surfaceOfAnnulus** : surface d'une couronne délimitée par deux cercles de rayons donnés

On peut utiliser

double Math.abs(double x)

et

Math.PI

Rédiger un programme `TestAnnulus` qui affiche la surface de la couronne délimitée par

- des cercles de rayons $r1 = 2.3$ et $r2 = 4.1$
- des cercles de rayons $r1 = 4.1$ et $r2 = 2.3$
- des cercles de rayons donnés en entrée

Exercice 4

On désire calculer le périmètre et l'aire de cercles et de rectangles. Pour des raisons de modularité, on décide de rédiger plusieurs classes :

- une classe **Circle**, spécialisée dans les calculs concernant les cercles,
- une classe **Rectangle**, spécialisée dans les calculs concernant les rectangles,
- une classe **TestModularity** contenant la procédure principale qui teste les deux classes précédentes pour un cercle de rayon 12 et pour un rectangle de côtés 12 et 15.

Définir ces trois classes.

Exercice 5

Rédiger et tester les fonctions suivantes :

1. Fonction **max** qui calcule le plus grand de deux entiers donnés en paramètres.
2. Fonction “maximum de trois nombres” qui calcule le plus grand de trois entiers donnés en paramètres. En réaliser deux versions :
 - version1 : **maxOfThreeNumbersV1**, en utilisant **max**,
 - version2 : **maxOfThreeNumbersV2**, directement, sans utiliser **max**.

Exercice 6

Rédiger une fonction qui calcule la médiane de trois entiers distincts, c'est-à-dire celui qui est “encadré” par les deux autres.

Exemple : la médiane de 8,1,4 est 4.

Exercice 7

Rédiger un programme qui affiche la ou les solutions d'une équation de second degré :

$$ax^2 + bx + c = 0$$

Attention : faire une analyse sérieuse. Les nombres a, b, c sont des nombres quelconques. Ils peuvent notamment être nuls.

*Conseil : rédiger une **fonction** qui rende en résultat une chaîne de caractères signifiant «en clair» le nombre et la nature des solutions.*

Exercice 8

Étant donné un entier $n \geq 0$, on souhaite trouver le i -ème chiffre de son écriture décimale. Les chiffres sont numérotés à partir de 0 depuis les poids faibles. Si i est plus grand que le rang du plus fort chiffre significatif, le résultat sera 0.

Exemple : pour $n = 245$

digit(0,n) vaut 5

digit(1,n) vaut 4

digit(2,n) vaut 2

digit(3,n) vaut 0, ...

Écrire la fonction **digit**. On pourra écrire deux versions de cette fonction : une version itérative, et une version récursive.

Exercice 9

Écrire la fonction **sumOfDigits** qui vaut la somme des chiffres de l'écriture décimale d'un entier positif ou nul. On pourra écrire deux versions de cette fonction : une version itérative, et une version récursive.

Exercice 10

La fameuse suite de Fibonacci est définie par la relation suivante :

$\text{fib}(0) = 1$

$\text{fib}(1) = 1$

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ pour $n > 1$

Rédiger une fonction **int fibo(int n)** qui calcule le n -ième terme de cette suite. On définira deux versions pour cette fonction : une version récursive et une version itérative.

Exercice 11

Rédiger un programme qui lit une suite de nombres entiers positifs, terminée par -1 et affiche la moyenne de ces nombres.

Exemple : Pour la suite d'entiers 4 3 0 4 -1, la moyenne est 2.75.

Exercice 12

On entre au clavier une suite non vide d'entiers positifs terminée par la marque -1. Rédiger un programme qui lit ces nombres et calcule la longueur de la plus longue sous-suite constante.

Exemple : Pour la suite d'entiers 7 5 5 4 1 1 1 1 8 8 4 1 -1, la longueur de la plus longue sous-suite constante est 4 (suite de quatre 1).

Exercice 13

Rédiger un programme qui détermine et affiche le nombre de mots contenus dans une phrase terminée par un point. On considère que le caractère séparateur de mots est le caractère espace. Attention, les espaces sont possibles avant ou après le premier mot, plusieurs espaces consécutifs sont aussi possibles.

Exercice 14

Ecrire la fonction `indexOfMax` qui rend un indice contenant la valeur maximale d'un tableau d'entiers.

Exercice 15

Soient deux tableaux X1 et X2 de nombres entiers triés par ordre croissant. Réunir les deux tableaux en un seul tableau trié.

Exercice 16

Un nombre complexe peut être représenté au moyen d'une structure (pas une "vraie" classe) à deux champs, x (partie réelle) et y (partie imaginaire).

Définir cette représentation au moyen d'une classe `Complex`.

— Rédiger la fonction statique `add` qui rend en résultat la somme de deux complexes.

- Rédiger la fonction statique `mul` qui rend en résultat le produit de deux complexes.
- Rédiger une fonction
`static String toString(Complex z)`
qui rend en résultat la chaîne de caractères de la forme `"x+y.i"` qui dénote le nombre complexe z sous la forme usuelle. On fera en sorte de rendre simplement `"x"` pour `"x+0.i"`, `"y.i"` pour `"0+y.i"`, `"x+i"` pour `"x+1.i"`, `"i"` pour `"0+i"` et `"0"` pour `"0+0.i"`.
- Rédiger une procédure principale qui lit deux réels x et y , crée le nombre complexe $z=x+y.i$ et calcule et affiche le nombre complexe $z.(z.(z+1)+1)$. Remarque : il faudra définir le complexe `"1"` (alias `"1+0.i"`) pour le faire participer aux opérations.

Exercice 17

A l'exercice précédent, nous avons représenté les nombres complexes au moyen d'une structure. Ce n'est pas vraiment la bonne façon de faire. Il faut totalement abstraire les données concrètes qui représentent un nombre complexe (x et y dans cet exemple), rédiger les fonctions de manipulation des complexes à l'intérieur de cette classe et ne rendre visible aux utilisateurs que ces fonctions.

Rédiger la classe `Complex` dans cet esprit.

Pour satisfaire aux goûts divers et variés des utilisateurs potentiels, on pourra réaliser deux versions de chaque fonction de calcul :

1. une version fonction statique, par exemple :

```
public static Complex add(Complex z1, Complex z2)
// résultat : la somme de z1 et de z2
```

2. une version méthode d'objet :

```
public Complex add(Complex z)
// résultat : la somme de this et de z
```